VGG Architectures on CIFAR-10: A Reduced-Scale Study of Depth and Batch Normalization

Student research report | PyTorch | CIFAR-10

Abstract— This project compares VGG-11, VGG-13, and VGG-16 on CIFAR-10, each with and without Batch Normalization (BN). It rebuilds the VGG idea of stacking small 3×3 convolutions in a reduced-scale setting. Six configurations were trained for five epochs using SGD with momentum and a cosine learning-rate schedule. The highest recorded test accuracy was 77.71% for VGG-11 without BN. Deeper BN variants showed unexpectedly low accuracy in these short runs. Because the training budget is limited, these are preliminary observations, not evidence that BN is generally detrimental. This work is inspired by the VGG paper but does not reproduce its original ImageNet experiment.

Index Terms— VGG, CNN, CIFAR-10, Batch Normalization, ablation, PyTorch.

I. INTRODUCTION

Convolutional neural networks (CNNs) learn image features through convolutional filters and nonlinear activations. Simonyan and Zisserman studied the effect of depth using networks built from repeated 3×3 filters. Their VGG architectures are influential because the design is regular and provides a clear way to study deeper convolutional stacks [1].

This project implements VGG-style networks in PyTorch and evaluates them on CIFAR-10, which is substantially smaller than ImageNet. The questions are: how does measured performance vary across three depths, and what changes when BN is switched on or off? The work is a reduced-scale study because the dataset, classifier head, and training budget differ from the original paper.

II. METHOD

A. Dataset

CIFAR-10 contains 60,000 RGB images of size 32×32 in ten classes. The notebook splits the original training set into 45,000 training and 5,000 validation images, and evaluates on the 10,000-image test set. The classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

B. VGG implementation

The models use repeated 3×3 convolutional layers, ReLU activations, and 2×2 max pooling. BN is the ablated component. The original ImageNet classifier is adapted for ten CIFAR-10 classes. The convolution parameter count for a 3×3 layer with C_{in} input channels and C_{out} output channels is:

$$N_{conv} = 3 \times 3 \times C_{in} \times C_{out} + C_{out}$$

Stacking two 3×3 convolutions has $18C^2$ weights for equal channel width C, compared with $25C^2$ for one 5×5 convolution, excluding biases. Stacking also introduces an additional nonlinear transformation. This is the central architectural motivation explored by VGG.

II. METHOD (CONTINUED)

C. Training configuration

The recorded setup uses CrossEntropyLoss, SGD with learning rate 0.05, momentum 0.9, weight decay 0.0005, Kaiming initialization, and CosineAnnealingLR. The notebook sets a random seed of 42. Each final reported configuration was trained for five epochs. Five epochs keeps the run manageable on a Colab T4 but is not enough to establish convergence. A final reproducibility pass should also record library versions, batch size, augmentation, and any runtime changes.

D. Ablation design

The primary ablation toggles BN while keeping the nominal architecture, dataset split, optimizer, and epoch budget fixed. This is intended to measure the effect of BN for each depth. Interpretation assumes the code paths are otherwise equivalent and the same training protocol is used.

III. RESULTS

A. Test accuracy and parameter count

Model	BN	Params	Test
VGG11	No	9,225,610	77.71%
VGG11	Yes	9,231,114	68.96%
VGG13	No	9,410,122	75.11%
VGG13	Yes	9,416,010	39.79%
VGG16	No	14,719,818	70.85%
VGG16	Yes	14,728,266	33.32%

VGG-11 without BN has the highest recorded test accuracy (77.71%). Among the no-BN configurations, the measured accuracy falls with increased depth in this particular run. The BN variants are lower, especially VGG-13 and VGG-16. This pattern is specific to the current training setup and should not be generalized to all implementations.

B. Batch Normalization ablation

The BN-minus-no-BN test accuracy differences are −8.75 percentage points for VGG-11, −35.32 for VGG-13, and −37.53 for VGG-16. These large differences are surprising enough to require diagnostic work. Possible causes include insufficient epochs, learning-rate interactions, BN placement, train/eval mode mistakes, or a mismatch in the actual training pipeline. The current results do not establish which explanation is correct.

IV. RELATION TO THE ORIGINAL PAPER

The original VGG paper evaluates deeper networks on large-scale ImageNet and reports results for 16–19 weight-layer architectures [1]. This project uses CIFAR-10, 32×32 inputs, a CIFAR-10-specific classifier, and five epochs. Consequently, the experiment does not numerically reproduce an ImageNet table. It tests the same broad architectural idea—stacking small filters and varying depth—under a constrained compute budget. A rigorous comparison should explicitly identify these differences rather than presenting the numbers as directly comparable.

V. FAILURE CASES AND LIMITATIONS

The main failure case is the low measured accuracy of deeper BN variants. Under-training or an implementation/training configuration issue may explain this; BN itself should not be declared harmful based on one short run. Recommended checks are: verify BN appears in the intended order relative to convolution and ReLU; ensure model.train() is used during training and model.eval() for validation/testing; inspect losses and gradients; train longer; and repeat runs with multiple seeds.

Other limitations include the short five-epoch budget, one recorded run per configuration, and the difference from the original ImageNet setup. The VGG-11 no-BN classification report also shows class-level variation: cat and bird are more difficult than automobile and ship in the recorded evaluation. Overall accuracy alone therefore does not capture all model behavior.

VI. CONCLUSION

The project implements VGG-style models in PyTorch and compares three depths with and without BN on CIFAR-10. The best recorded test accuracy is 77.71% for VGG-11 without BN. The BN ablation reveals unexpectedly poor deeper-model results, motivating longer training and implementation checks. The notebook and preliminary experiments provide a foundation, but a stronger reproduction requires rerunning the experiments under a documented protocol and explicitly comparing the reduced-scale setup with the original paper.

REFERENCES

[1] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” arXiv:1409.1556, 2014 (revised 2015). https://arxiv.org/abs/1409.1556

[2] PyTorch documentation, “torch.nn” and TorchVision CIFAR-10 dataset documentation. https://pytorch.org/docs/stable/ and https://pytorch.org/vision/stable/

APPENDIX: REPRODUCIBILITY CHECKLIST

Before submitting, rerun the notebook from a clean Colab runtime and verify all cells execute in order. Confirm the seed, data split, model configuration, BN placement, optimizer, scheduler, and epoch count. Export the final result tables and plots from that run, then update the README and report if the results change. If a third-party implementation was consulted, cite its URL and clearly identify what was reused.

The five-epoch results in this report are the values provided in the project conversation. The report does not claim that the experiments were rerun while preparing this document. Any final submission should include the student's own execution evidence and should disclose changes made after these recorded runs.